

async_scope – Creating scopes for non-sequential concurrency

Document #: P3149R6
Date: 2024-10-15
Project: Programming Language C++
Audience: SG1 Parallelism and Concurrency
LEWG Library Evolution
Reply-to: Ian Petersen
<ispeters@gmail.com>
Jessica Wong
<jesswong2011@gmail.com>
Contributors: Ján Ondrušek
<ondrusek@meta.com>
Kirk Shoop
<kirk.shoop@gmail.com>
Lee Howes
<lwh@fb.com>
Lucian Radu Teodorescu
<lucteo@lucteo.ro>

Contents

- 1 Changes
 - 1.1 R6
 - 1.1.1 The Problem
 - 1.1.2 Some Solutions
 - 1.1.3 LEWG Discussion in St Louis
 - 1.2 R5
 - 1.3 R4
 - 1.4 R3
 - 1.5 R2
 - 1.6 R1
 - 1.7 R0
- 2 Introduction
 - 2.1 Implementation experience
- 3 Motivation
 - 3.1 Motivating example
 - 3.2 `counting_scope` and `let_async_scope` are a step forward towards Structured Concurrency
- 4 Examples of use
 - 4.1 Spawning work from within a task
 - 4.2 Starting work nested within a framework
 - 4.3 Starting parallel work
 - 4.4 Listener loop in an HTTP server
 - 4.5 Pluggable functionality through composition
 - 4.6 Recursively spawning work until completion
 - 4.6.1 `let_async_scope` with `spawn()`
 - 4.6.2 `let_async_scope` with `spawn_future()`
 - 4.6.3 `counting_scope`
- 5 Async Scope, usage guide

- 5.1 Definitions
- 5.2 `execution::async_scope_association`
- 5.3 `execution::async_scope_token`
- 5.4 `execution::nest`
- 5.5 `execution::spawn`
- 5.6 `execution::spawn_future`
- 5.7 `execution::simple_counting_scope`
 - 5.7.1 `simple_counting_scope::simple_counting_scope`
 - 5.7.2 `simple_counting_scope::~simple_counting_scope`
 - 5.7.3 `simple_counting_scope::get_token`
 - 5.7.4 `simple_counting_scope::close`
 - 5.7.5 `simple_counting_scope::join`
 - 5.7.6 `simple_counting_scope::assoc::assoc`
 - 5.7.7 `simple_counting_scope::assoc::~assoc`
 - 5.7.8 `simple_counting_scope::assoc::operator=`
 - 5.7.9 `simple_counting_scope::assoc::operator bool`
 - 5.7.10 `simple_counting_scope::token::wrap`
 - 5.7.11 `simple_counting_scope::token::try_associate`
- 5.8 `execution::counting_scope`
 - 5.8.1 `counting_scope::counting_scope`
 - 5.8.2 `counting_scope::~counting_scope`
 - 5.8.3 `counting_scope::get_token`
 - 5.8.4 `counting_scope::close`
 - 5.8.5 `counting_scope::request_stop`
 - 5.8.6 `counting_scope::join`
 - 5.8.7 `counting_scope::token::wrap`
 - 5.8.8 `counting_scope::token::try_associate`
- 5.9 When to use `counting_scope` vs [P3296R2]’s `let_async_scope`
- 6 Design considerations
 - 6.1 Shape of the given sender
 - 6.1.1 Constraints on `set_value()`
 - 6.1.2 Handling errors in `spawn()`
 - 6.1.3 Handling stop signals in `spawn()`
 - 6.1.4 No shape restrictions for the senders passed to `spawn_future()` and `nest()`
 - 6.2 P2300’s `start_detached()`
 - 6.3 P2300’s `ensure_started()`
 - 6.4 Supporting the pipe operator
- 7 Naming
 - 7.1 `async_scope_token`
 - 7.2 `nest()`
 - 7.3 `spawn()`
 - 7.4 `spawn_future()`
 - 7.5 `simple_counting_scope`
 - 7.6 `counting_scope`
 - 7.6.1 `counting_scope::join()`

- 8 Specification
 - 8.1 Async scope concepts
 - 8.2 `execution::nest`
 - 8.3 `execution::spawn`
 - 8.4 `execution::spawn_future`
 - 8.5 `execution::simple_counting_scope`
 - 8.6 `execution::counting_scope`
- 9 Acknowledgements
- 10 References

§ 1 Changes

§ 1.1 R6

In revision 4 of this paper, Lewis Baker discovered a problem with using `nest()` as the basis operation for implementing `spawn()` (and `spawn_future()`) when the `counting_scope` that tracks the spawned work is being used to protect against out-of-lifetime accesses to the allocator provided to `spawn()`. Revision 5 of this paper raised Lewis’s concerns and presented several solutions. Revision 6 has selected the solution originally presented as “option 4”: define a new set of refcounting basis operations and define `nest()`, `spawn()`, and `spawn_future()` in terms of them.

§ 1.1.1 The Problem

What follows is a description, taken from revision 5, section 6.5.1, of the problem with using `nest()` as the basis operation for implementing `spawn()` (a similar problem exists for `spawn_future()` but `spawn()` is simpler to explain).

When a spawned operation completes, the order of operations was as follows:

1. The spawned operation completes by invoking `set_value()` or `set_stopped()` on a receiver, `rcvr`, provided by `spawn()` to the `nest`-sender.
2. `rcvr` destroys the `nest`-sender’s *operation-state* by invoking its destructor.
3. `rcvr` deallocates the storage previously allocated for the just-destroyed *operation-state* using a copy of the allocator that was chosen when `spawn()` was invoked. Assume this allocator was passed to `spawn()` in the optional environment argument.

Note that in step 2, above, the destruction of the `nest`-sender’s *operation-state* has the side effect of decrementing the associated `counting_scope`’s count of outstanding operations. If the scope has a `join`-sender waiting and this decrement brings the count to zero, the code waiting on the `join`-sender to complete may start to destroy the allocator while step 3 is busy using it.

§ 1.1.2 Some Solutions

Revision 5 presented the following possible solutions:

1. Do nothing; declare that `counting_scope` can’t be used to protect memory allocators.
2. Remove allocator support from `spawn()` and `spawn_future()` and require allocation with `::operator new`.
3. Make `spawn()` and `spawn_future()` basis operations of `async_scope_tokens` (alongside `nest()`) so that the decrement in step 2 can be deferred until after step 3 completes.
4. Define a new set of refcounting basis operations and define `nest()`, `spawn()`, and `spawn_future()` in terms of them.
5. Treat `nest`-senders as RAII handles to “scope references” and change how `spawn()` is defined to defer the decrement. (There are a few implementation possibilities here.)
6. Give `async_scope_tokens` a new basis operation that can wrap an allocator in a new allocator wrapper that increments the scope’s refcount in `allocate()` and decrements it in `deallocate()`.

§ 1.1.3 LEWG Discussion in St Louis

The authors opened the discussion by recommending option 6. By the end of the discussion, the authors’ expressed preferences were: “4 & 6 are better than 5; 5 is better than 3.” The biggest concern with option 4 was the time required to rework the paper in terms of the new basis operation.

The room took the following two straw polls:

1. In P3149R5 strike option 1 from 6.5.2 (option 1 would put the responsibility to coordinate the lifetime of the memory resource on the end user)

SF	F	N	A	SA
10	2	3	1	1

Attendance: 21 in-person + 10 remote

of Authors: 2

Authors' position: 2x SF

Outcome: Consensus in favor

SA: I'm SA because I don't think async scope needs to protect memory allocations or resources, it's fine for this not to be a capability and I think adding this capability will add complexity, and that'll mean it doesn't make C++26.

2. In P3149R5 strike option 2 from 6.5.2 (option 2 would prevent spawn from supporting allocators)

SF	F	N	A	SA
8	4	2	2	0

Attendance: 21 in-person + 10 remote

of Authors: 2

Authors' position: 2x SF

Outcome: Consensus in favor

WA: As someone who was weakly against I'm not ready to rule out this possibility yet.

Ultimately, the authors chose option 4, leading to revision 6 of the paper changing from this:

```
template <class Token, class Sender>
concept async_scope_token =
    copyable<Token> &&
    is_nothrow_move_constructible_v<Token> &&
    is_nothrow_move_assignable_v<Token> &&
    is_nothrow_copy_constructible_v<Token> &&
    is_nothrow_copy_assignable_v<Token> &&
    sender<Sender> &&
    requires(Token token, Sender&& snd) {
        { token.nest(std::forward<Sender>(snd)) } -> sender;
    };
```

with `execution::nest()` forwarding to the `nest()` method on the provided token and `spawn()` and `spawn_future()` being expressed in terms of `nest()`, to this:

```
template <class Assoc>
concept async_scope_association =
    semiregular<Assoc> &&
    requires(const Assoc& assoc) {
        { static_cast<bool>(assoc) } noexcept;
    };

template <class Token>
concept async_scope_token =
    copyable<Token> &&
    requires(Token token) {
        { token.try_associate() } -> async_scope_association;
    };
};
```

with `nest()`, `spawn()`, and `spawn_future()` all being expressed in terms of the `async_scope_token` concept.

§ 1.2 R5

- Clarify that the *nest-sender*'s operation state must destroy its child operation state before decrementing the scope's reference count.
- Add naming discussion.
- Discuss a memory allocator lifetime concern raised by Lewis Baker and several options for resolving it.

§ 1.3 R4

- Permit caller of `spawn_future()` to provide a stop token in the optional environment argument.
- Remove `[[nodiscard]]`.
- Make `simple_counting_scope::token::token()` and `counting_scope::token::token()` explicit and exposition-only.
- Remove redundant concept `async_scope`.
- Remove last vestiges of `let_async_scope`.
- Add some wording to a new [Specification](#) section

§ 1.4 R3

- Update slide code to be exception safe
- Split the `async_scope` concept into a `scope` and `token`; update `counting_scope` to match
- Rename `counting_scope` to `simple_counting_scope` and give the name `counting_scope` to a `scope` with a stop source
- Add example for recursively spawned work using `let_async_scope` and `counting_scope`

§ 1.5 R2

- Update `counting_scope::nest()` to explain when the scope's count of outstanding senders is decremented and remove `counting_scope::joined()`, `counting_scope::join_started()`, and `counting_scope::use_count()` on advice of SG1 straw poll:

forward P3149R1 to LEWG for inclusion in C++26 after P2300 is included in C++26, with notes:

1. the point of `refcount` decrement to be moved after the child operation state is destroyed
2. a future paper should explore the design for cancellation of scopes
3. observers (`joined`, `join_started`, `use_count`) can be removed

SF	F	N	A	SA
10	14	2	0	1

Consensus

SA: we are moving something without wide implementation experience, the version with experience has cancellation of scopes

- Add a fourth state to `counting_scope` so that it can be used as a data-member safely

§ 1.6 R1

- Add implementation experience
- Incorporate pre-meeting feedback from Eric Niebler

§ 1.7 R0

- First revision

§ 2 Introduction

[P2300R7] lays the groundwork for writing structured concurrent programs in C++ but it leaves three important scenarios under- or unaddressed:

1. progressively structuring an existing, unstructured concurrent program;
2. starting a dynamic number of parallel tasks without “losing track” of them; and
3. opting in to eager execution of sender-shaped work when appropriate.

This paper describes the utilities needed to address the above scenarios within the following constraints:

- *No detached work by default*; as specified in [P2300R7], the `start_detached` and `ensure_started` algorithms invite users to start concurrent work with no built-in way to know when that work has finished.

- Such so-called “detached work” is undesirable; without a way to know when detached work is done, it is difficult know when it is safe to destroy any resources referred to by the work. Ad hoc solutions to this shutdown problem add unnecessary complexity that can be avoided by ensuring all concurrent work is “attached”.
- [P2300R7]’s introduction of structured concurrency to C++ will make async programming with C++ much easier but experienced C++ programmers typically believe that async C++ is “just hard” and that starting async work *means* starting detached work (even if they are not thinking about the distinction between attached and detached work) so adapting to a post-[P2300R7] world will require unlearning many deprecated patterns. It is thus useful as a teaching aid to remove the unnecessary temptation of falling back on old habits.
- *No dependencies besides [P2300R7]*; it will be important for the success of [P2300R7] that existing code bases can migrate from unstructured concurrency to structured concurrency in an incremental way so tools for progressively structuring code should not take on risk in the form of unnecessary dependencies.

The proposed solution comes in the following parts:

- `template <class Assoc> concept async_scope_association;`
- `template <class Token> concept async_scope_token;`
- `sender auto nest(sender auto&& snd, async_scope_token auto token);`
- `void spawn(sender auto&& snd, async_scope_token auto token, auto&& env);`
- `sender auto spawn_future(sender auto&& snd, async_scope_token auto token, auto&& env);`
- Proposed in [P3296R2]: `sender auto let_async_scope(callable auto&& senderFactory);`
- `struct simple_counting_scope;` and
- `struct counting_scope.`

§ 2.1 Implementation experience

The general concept of an async scope to manage work has been deployed broadly at Meta. Code written with Folly’s coroutine library, [folly::coro], uses [folly::coro::AsyncScope] to safely launch awaitables. Most code written with Unifex, an implementation of an earlier version of the *Sender/Receiver* model proposed in [P2300R7], uses [unifex::v1::async_scope], although experience with the v1 design led to the creation of [unifex::v2::async_scope], which has a smaller interface and a cleaner definition of responsibility.

As an early adopter of Unifex, [rsys] (Meta’s cross-platform voip client library) became the entry point for structured concurrency in mobile code at Meta. We originally built rsys with an unstructured asynchrony model built around posting callbacks to threads in order to optimize for binary size. However, this came at the expense of developer velocity due to the increasing cost of debugging deadlocks and crashes resulting from race conditions.

We decided to adopt Unifex and refactor towards a more structured architecture to address these problems systematically. Converting an unstructured production codebase to a structured one is such a large project that it needs to be done in phases. As we began to convert callbacks to senders/tasks, we quickly realized that we needed a safe place to start structured asynchronous work in an unstructured environment. We addressed this need with unifex::v1::async_scope paired with an executor to address a recurring pattern:

Before	After
<pre> // Abstraction for thread that has the ability // to execute units of work. class Executor { public: virtual void add(Func function) noexcept = 0; }; // Example class class Foo { std::shared_ptr<Executor> exec_; public: void doSomething() { auto asyncWork = [&]() { // do something }; exec_>add(asyncWork); } }; </pre>	<pre> // Utility class for executing async work on an // async_scope and on the provided executor class ExecutorAsyncScopePair { unifex::v1::async_scope scope_; ExecutorScheduler exec_; public: void add(Func func) { scope_.detached_spawn_call_on(exec_, func); } auto cleanup() { return scope_.cleanup(); } }; // Example class class Foo { std::shared_ptr<ExecutorAsyncScopePair> exec_; public: ~Foo() { sync_wait(exec_>cleanup()); } }; </pre>

Before	After
	<pre> void doSomething() { auto asyncWork = [&]() { // do something }; exec_-->add(asyncWork); } }; </pre>

This broadly worked but we discovered that the above design coupled with the v1 API allowed for too many redundancies and conflated too many responsibilities (scoping async work, associating work with a stop source, and transferring scoped work to a new scheduler).

We learned that making each component own a distinct responsibility will minimize the confusion and increase the structured concurrency adoption rate. The above example was an intuitive use of `async_scope` because the concept of a “scoped executor” was familiar to many engineers and is a popular async pattern in other programming languages. However, the above design abstracted away some of the APIs in `async_scope` that explicitly asked for a scheduler, which would have helped challenge the assumption engineers made about `async_scope` being an instance of a “scoped executor”.

Cancellation was an unfamiliar topic for engineers within the context of asynchronous programming. The `v1::async_scope` provided both `cleanup()` and `complete()` to give engineers the freedom to decide between canceling work or waiting for work to finish. The different nuances on when this should happen and how it happens ended up being an obstacle that engineers didn’t want to deal with.

Over time, we also found redundancies in the way `v1::async_scope` and other algorithms were implemented and identified other use cases that could benefit from a different kind of async scope. This motivated us to create `v2::async_scope` which only has one responsibility (scope), and `nest` which helped us improve maintainability and flexibility of Unifex.

The unstructured nature of `cleanup()/complete()` in a partially structured codebase introduced deadlocks when engineers nested the `cleanup()/complete()` sender in the scope being joined. This risk of deadlock remains with `v2::async_scope::join()` however, we do think this risk can be managed and is worth the tradeoff in exchange for a more coherent architecture that has fewer crashes. For example, we have experienced a significant reduction in these types of deadlocks once engineers understood that `join()` is a destructor-like operation that needs to be run only by the scope’s owner. Since there is no language support to manage async lifetimes automatically, this insight was key in preventing these types of deadlocks. Although this breakthrough was a result of strong guidance from experts, we believe that the simpler design of `v2::async_scope` would make this a little easier.

We strongly believe that `async_scope` was necessary for making structured concurrency possible within `rsys`, and we believe that the improvements we made with `v2::async_scope` will make the adoption of P2300 more accessible.

§ 3 Motivation

§ 3.1 Motivating example

Let us assume the following code:

```

namespace ex = std::execution;

struct work_context;
struct work_item;
void do_work(work_context&, work_item*);
std::vector<work_item*> get_work_items();

int main() {
    static_thread_pool my_pool{8};
    work_context ctx; // create a global context for the application

    std::vector<work_item*> items = get_work_items();
    for (auto item : items) {
        // Spawn some work dynamically
        ex::sender auto snd = ex::transfer_just(my_pool.get_scheduler(), item) |
                               ex::then([&](work_item* item) { do_work(ctx, item); });
        ex::start_detached(std::move(snd));
    }
    // `ctx` and `my_pool` are destroyed
}

```

In this example we are creating parallel work based on the given input vector. All the work will be spawned on the local `static_thread_pool` object, and will use a shared `work_context` object.

Because the number of work items is dynamic, one is forced to use `start_detached()` from [P2300R7] (or something equivalent) to dynamically spawn work. [P2300R7] doesn't provide any facilities to spawn dynamic work and return a sender (i.e., something like `when_all` but with a dynamic number of input senders).

Using `start_detached()` here follows the *fire-and-forget* style, meaning that we have no control over, or awareness of, the completion of the async work that is being spawned.

At the end of the function, we are destroying the `work_context` and the `static_thread_pool`. But at that point, we don't know whether all the spawned async work has completed. If any of the async work is incomplete, this might lead to crashes.

[P2300R7] doesn't give us out-of-the-box facilities to use in solving these types of problems.

This paper proposes the `counting_scope` and [P3296R2]'s `let_async_scope` facilities that would help us avoid the invalid behavior. With `counting_scope`, one might write safe code this way:

```
namespace ex = std::execution;

struct work_context;
struct work_item;
void do_work(work_context&, work_item*);
std::vector<work_item*> get_work_items();

int main() {
    static_thread_pool my_pool{8};
    work_context ctx;           // create a global context for the application
    ex::counting_scope scope;    // create this *after* the resources it protects

    // make sure we always join
    unifex::scope_guard join = [&]() noexcept {
        // wait for all nested work to finish
        this_thread::sync_wait(scope.join()); // NEW!
    };

    std::vector<work_item*> items = get_work_items();
    for (auto item : items) {
        // Spawn some work dynamically
        ex::sender auto snd = ex::transfer_just(my_pool.get_scheduler(), item) |
            ex::then([&](work_item* item) { do_work(ctx, item); });

        // start `snd` as before, but associate the spawned work with `scope` so that it can
        // be awaited before destroying the resources referenced by the work (i.e. `my_pool`
        // and `ctx`)
        ex::spawn(std::move(snd), scope.get_token()); // NEW!
    }

    // `ctx` and `my_pool` are destroyed *after* they are no longer referenced
}
```

With [P3296R2]'s `let_async_scope`, one might write safe code this way:

```
namespace ex = std::execution;

struct work_context;
struct work_item;
void do_work(work_context&, work_item*);
std::vector<work_item*> get_work_items();

int main() {
    static_thread_pool my_pool{8};
    work_context ctx; // create a global context for the application

    this_thread::sync_wait(
        ex::let_async_scope(ex::just(get_work_items()), [&](auto scope, auto& items) {
            for (auto item : items) {
                // Spawn some work dynamically
                ex::sender auto snd = ex::transfer_just(my_pool.get_scheduler(), item) |
                    ex::then([&](work_item* item) { do_work(ctx, item); });

                // start `snd` as before, but associate the spawned work with `scope` so that it
                // can be awaited before destroying the resources referenced by the work (i.e.
                // `my_pool` and `ctx`)
                ex::spawn(std::move(snd), scope); // NEW!
            }
            return just();
        }));
}
```



```
// `ctx` and `my_pool` are destroyed after they are no longer referenced
}
```

Simplifying the above into something that fits in a Tony Table to highlight the differences gives us:

Before	With <code>counting_scope</code>	With <code>let_async_scope</code>
<pre>namespace ex = std::execution; struct context; ex::sender auto work(const context&); int main() { context ctx; ex::sender auto snd = work(ctx); // fire and forget ex::start_detached(std::move(snd)); // `ctx` is destroyed, perhaps // before // `snd` is done }</pre>	<pre>namespace ex = std::execution; struct context; ex::sender auto work(const context&); int main() { context ctx; ex::counting_scope scope; ex::sender auto snd = work(ctx); try { // fire, but don't forget ex::spawn(std::move(snd), scope.get_token()); } catch (...) { // do something to handle // exception } // wait for all work nested within // scope // to finish this_thread::sync_wait(scope.join()); // `ctx` is destroyed once nothing // references it }</pre>	<pre>namespace ex = std::execution; struct context; ex::sender auto work(const context&); int main() { context ctx; this_thread::sync_wait(ex::just() ex::let_async_scope([& (auto scope) { ex::sender auto snd = work(ctx); // fire, but don't forget ex::spawn(std::move(snd), scope.get_token()); })); // `ctx` is destroyed once // nothing // references it }</pre>

Please see below for more examples.

§ 3.2 `counting_scope` and `let_async_scope` are a step forward towards Structured Concurrency

Structured Programming [Dahl72] transformed the software world by making it easier to reason about the code, and build large software from simpler constructs. We want to achieve the same effect on concurrent programming by ensuring that we *structure* our concurrent code. [P2300R7] makes a big step in that direction, but, by itself, it doesn't fully realize the principles of Structured Programming. More specifically, it doesn't always ensure that we can apply the *single entry, single exit point* principle.

The `start_detached` sender algorithm fails this principle by behaving like a GOTO instruction. By calling `start_detached` we essentially continue in two places: in the same function, and on different thread that executes the given work. Moreover, the lifetime of the work started by `start_detached` cannot be bound to the local context. This will prevent local reasoning, which will make the program harder to understand.

To properly structure our concurrency, we need an abstraction that ensures that all async work that is spawned has a defined, observable, and controllable lifetime. This is the goal of `counting_scope` and `let_async_scope`.

§ 4 Examples of use

§ 4.1 Spawning work from within a task

Use `let_async_scope` in combination with a `system_context` from [P2079R2] to spawn work from within a task:

```
namespace ex = std::execution;

int main() {
    ex::system_context ctx;
    int result = 0;

    ex::scheduler auto sch = ctx.scheduler();

    ex::sender auto val = ex::just() | ex::let_async_scope([sch](ex::async_scope_token auto scope) {
        int val = 13;

        auto print_sender = ex::just() | ex::then([val]() noexcept {
```

```

        std::cout << "Hello world! Have an int with value: " << val << "\n";
    });

    // spawn the print sender on sch
    //
    // NOTE: if spawn throws, let_async_scope will capture the exception
    //        and propagate it through its set_error completion
    ex::spawn(ex::on(sch, std::move(print_sender)), scope);

    return ex::just(val);
} | ex::then([&result](auto val) { result = val });

this_thread::sync_wait(ex::on(sch, std::move(val)));

std::cout << "Result: " << result << "\n";
}

// 'let_async_scope' ensures that, if all work is completed successfully, the result will be 13
// 'sync_wait' will throw whatever exception is thrown by the callable passed to 'let_async_scope'

```

§ 4.2 Starting work nested within a framework

In this example we use the `counting_scope` within a class to start work when the object receives a message and to wait for that work to complete before closing.

```

namespace ex = std::execution;

struct my_window {
    class close_message {};

    ex::sender auto some_work(int message);

    ex::sender auto some_work(close_message message);

    void onMessage(int i) {
        ++count;
        ex::spawn(ex::on(sch, some_work(i)), scope);
    }

    void onClickClose() {
        ++count;
        ex::spawn(ex::on(sch, some_work(close_message{})), scope);
    }

    my_window(ex::system_scheduler sch, ex::counting_scope::token scope)
        : sch(sch)
        , scope(scope) {
        // register this window with the windowing framework somehow so that
        // it starts receiving calls to onClickClose() and onMessage()
    }

    ex::system_scheduler sch;
    ex::counting_scope::token scope;
    int count{0};
};

int main() {
    // keep track of all spawned work
    ex::counting_scope scope;
    ex::system_context ctx;
    try {
        my_window window{ctx.get_scheduler(), scope.get_token()};
    } catch (...) {
        // do something with exception
    }
    // wait for all work nested within scope to finish
    this_thread::sync_wait(scope.join());
    // all resources are now safe to destroy
    return window.count;
}

```

§ 4.3 Starting parallel work

In this example we use `let_async_scope` to construct an algorithm that performs parallel work. Here `foo` launches 100 tasks that concurrently run on some scheduler provided to `foo`, through its connected receiver, and then the tasks are asynchronously joined. This structure emulates how we might build a parallel algorithm where each `some_work` might be operating on a fragment of data.

```

namespace ex = std::execution;

ex::sender auto some_work(int work_index);

ex::sender auto foo(ex::scheduler auto sch) {
    return ex::just() | ex::let_async_scope([sch](ex::async_scope_token auto scope) {
        return ex::schedule(sch) | ex::then([] { std::cout << "Before tasks launch\n"; }) |
            ex::then([=] {
                // Create parallel work
                for (int i = 0; i < 100; ++i) {
                    // NOTE: if spawn() throws, the exception will be propagated as the
                    //       result of let_async_scope through its set_error completion
                    ex::spawn(ex::on(sch, some_work(i)), scope);
                }
            });
    }) | ex::then([] { std::cout << "After tasks complete successfully\n"; });
}

```

§ 4.4 Listener loop in an HTTP server

This example shows how one can write the listener loop in an HTTP server, with the help of coroutines. The HTTP server will continuously accept new connection and start work to handle the requests coming on the new connections. While the listening activity is bound in the scope of the loop, the lifetime of handling requests may exceed the scope of the loop. We use `counting_scope` to limit the lifetime of the request handling without blocking the acceptance of new requests.

```

namespace ex = std::execution;

task<size_t> listener(int port, io_context& ctx, static_thread_pool& pool) {
    size_t count{0};
    listening_socket listen_sock{port};

    co_await ex::let_async_scope(ex::just(), [&](ex::async_scope_token auto scope) -> task<void> {
        while (!ctx.is_stopped()) {
            // Accept a new connection
            connection conn = co_await async_accept(ctx, listen_sock);
            count++;

            // Create work to handle the connection in the scope of `work_scope`
            conn_data data{std::move(conn), ctx, pool};
            ex::sender auto snd = ex::just(std::move(data)) |
                ex::let_value([](auto& data) { return handle_connection(data); });

            ex::spawn(std::move(snd), scope);
        }
    });

    // At this point, all the request handling is complete
    co_return count;
}

```

[[libunifex](#)] has a very similar example HTTP server at [[io_uring HTTP server](#)] that compiles and runs on Linux-based machines with `io_uring` support.

§ 4.5 Pluggable functionality through composition

This example is based on real code in `rsys`, but it reduces the real code to slideware and ports it from Unifex to the proposed `std::execution` equivalents. The central abstraction in `rsys` is a `Call`, but each integration of `rsys` has different needs so the set of features supported by a `Call` varies with the build configuration. We support this configurability by exposing the equivalent of the following method on the `Call` class:

```

template <typename Feature>
Handle<Feature> Call::get();

```

and it's used like this in app-layer code:

```

unifex::task<void> maybeToggleCamera(Call& call) {
    Handle<Camera> camera = call.get<Camera>();

    if (camera) {
        co_await camera->toggle();
    }
}

```

A `Handle<Feature>` is effectively a part-owner of the `Call` it came from.

The team that maintains rsys and the teams that use rsys are, unsurprisingly, different teams so rsys has to be designed to solve organizational problems as well as technical problems. One relevant design decision the rsys team made is that it is safe to keep using a `Handle<Feature>` after the end of its `Call`'s lifetime; this choice adds some complexity to the design of `Call` and its various features but it also simplifies the support relationship between the rsys team and its many partner teams because it eliminates many crash-at-shutdown bugs.

```
namespace rsys {

class Call {
public:
    unifex::nothrow_task<void> destroy() noexcept {
        // first, close the scope to new work and wait for existing work to finish
        scope_>close();
        co_await scope_>join();

        // other clean-up tasks here
    }

    template <typename Feature>
    Handle<Feature> get() noexcept;

private:
    // an async scope shared between a call and its features
    std::shared_ptr<std::execution::counting_scope> scope_;
    // each call has its own set of threads
    ExecutionContext context_;

    // the set of features this call supports
    FeatureBag features_;
};

class Camera {
public:
    std::execution::sender auto toggle() {
        namespace ex = std::execution;

        return ex::just() | ex::let_value([this]() {
            // this callable is only invoked if the Call's scope is in
            // the open or unused state when nest() is invoked, making
            // it safe to assume here that:
            //
            // - scheduler_ is not a dangling reference to the call's
            //   execution context
            // - Call::destroy() has not progressed past starting the
            //   join-sender so all the resources owned by the call
            //   are still valid
            //
            // if the nest() attempt fails because the join-sender has
            // started (or even if the Call has been completely destroyed)
            // then the sender returned from toggle() will safely do
            // nothing before completing with set_stopped()

            return ex::schedule(scheduler_) | ex::then([this]() {
                // toggle the camera
            });
        }) | ex::nest(callScope_>get_token());
    }

private:
    // a copy of this camera's Call's scope_member
    std::shared_ptr<ex::counting_scope> callScope_;
    // a scheduler that refers to this camera's Call's ExecutionContext
    Scheduler scheduler_;
};

} // namespace rsys
```

§ 4.6 Recursively spawning work until completion

Below are three ways you could recursively spawn work on a scope using `let_async_scope` or `counting_scope`.

§ 4.6.1 `let_async_scope` with `spawn()`

```
struct tree {
    std::unique_ptr<tree> left;
    std::unique_ptr<tree> right;
```

```

    int data;
};

auto process(ex::scheduler auto sch, auto scope, tree& t) noexcept {
    return ex::schedule(sch) | then([sch, &t]() {
        if (t.left)
            ex::spawn(process(sch, scope, t.left.get()), scope);
        if (t.right)
            ex::spawn(process(sch, scope, t.right.get()), scope);
        do_stuff(t.data);
    }) | ex::let_error([](auto& e) {
        // log error
        return just();
    });
}

int main() {
    ex::scheduler sch;
    tree t = make_tree();
    // let_async_scope will ensure all new work will be spawned on the
    // scope and will not be joined until all work is finished.
    // NOTE: Exceptions will not be surfaced to let_async_scope; exceptions
    // will be handled by let_error instead.
    this_thread::sync_wait(ex::just() | ex::let_async_scope([&, sch](auto scope) {
        return process(sch, scope, t);
    }));
}

```

§ 4.6.2 let_async_scope with spawn_future()

```

struct tree {
    std::unique_ptr<tree> left;
    std::unique_ptr<tree> right;
    int data;
};

auto process(ex::scheduler auto sch, auto scope, tree& t) {
    return ex::schedule(sch) | ex::let_value([sch, &t]() {
        unifex::any_sender_of<> leftFut = ex::just();
        unifex::any_sender_of<> rightFut = ex::just();
        if (t.left) {
            leftFut = ex::spawn_future(scope, process(sch, scope, t.left.get()));
        }

        if (t.right) {
            rightFut = ex::spawn_future(scope, process(sch, scope, t.right.get()));
        }

        do_stuff(t.data);
        return ex::when_all(leftFut, rightFut) | ex::then([](auto&&...) noexcept {});
    });
}

int main() {
    ex::scheduler sch;
    tree t = make_tree();
    // let_async_scope will ensure all new work will be spawned on the
    // scope and will not be joined until all work is finished
    // NOTE: Exceptions will be surfaced to let_async_scope which will
    // call set_error with the exception_ptr
    this_thread::sync_wait(ex::just() | ex::let_async_scope([&, sch](auto scope) {
        return process(sch, scope, t);
    }));
}

```

§ 4.6.3 counting_scope

```

struct tree {
    std::unique_ptr<tree> left;
    std::unique_ptr<tree> right;
    int data;
};

auto process(ex::counting_scope_token scope, ex::scheduler auto sch, tree& t) noexcept {
    return ex::schedule(sch) | ex::then([sch, &t]() noexcept {
        if (t.left)
            ex::spawn(process(scope, sch, t.left.get()), scope);
    });
}

```

```

        if (t.right)
            ex::spawn(process(scope, sch, t.right.get()), scope);

        do_stuff(t.data);
    } | ex::let_error([](auto& e) {
        // log error
        return just();
    });
}

int main() {
    ex::scheduler sch;
    tree t = make_tree();
    ex::counting_scope scope;
    ex::spawn(process(scope.get_token(), sch, t), scope.get_token());
    this_thread::sync_wait(scope.join());
}

```

§ 5 Async Scope, usage guide

An async scope is a type that implements a “bookkeeping policy” for senders that have been associated with the scope. Depending on the policy, different guarantees can be provided in terms of the lifetimes of the scope and any associated senders. The `counting_scope` described in this paper defines a policy that has proven useful while progressively adding structure to existing, unstructured code at Meta, but other useful policies are possible. By defining `nest()`, `spawn()`, and `spawn_future()` in terms of the more fundamental async scope token interface, and leaving the implementation of the abstract interface to concrete token types, this paper’s design leaves the set of policies open to extension by user code or future standards.

An async scope token’s implementation of the `async_scope_token` concept:

- must allow an arbitrary sender to be wrapped without eagerly starting the sender;
- must not add new value or error completions when wrapping a sender;
- may fail to associate a new sender by returning an association that converts to `false` from `try_associate()`;
- may fail to associate a new sender by eagerly throwing an exception from either `try_associate()` or `wrap()`;

More on these items can be found below in the sections below.

§ 5.1 Definitions

```

namespace { // exposition-only

template <class Env>
struct spawn-env; // exposition-only

template <class Env>
struct spawn-receiver { // exposition-only
    void set_value() noexcept;
    void set_stopped() noexcept;

    const spawn-env<Env>& get_env() const noexcept;
};

template <class Env>
struct future-env; // exposition-only

template <valid-completion-signatures Sigs>
struct future-sender; // exposition-only

template <sender Sender, class Env>
using future-sender-t = // exposition-only
    future-sender<completion_signatures_of_t<Sender, future-env<Env>>>;

}

template <class Assoc>
concept async_scope_association =
    semiregular<Assoc> &&
    requires(const Assoc& assoc) {
        { static_cast<bool>(assoc) } noexcept;
    };

template <class Token>
concept async_scope_token =
    copyable<Token> &&

```

```

    requires(Token token) {
        { token.try_associate() } -> async_scope_association;
    };

template <async_scope_token Token>
using association-from = decltype(declval<Token&>().try_associate()); // exposition-only

template <async_scope_token Token, sender Sender>
using wrapped-sender-from = decay_t<decltype(declval<Token&>().wrap(declval<Sender>()))>; // @@_exposition-only_@

template <sender Sender, async_scope_token Token>
struct nest-sender { // exposition-only
    nest-sender(Sender&& sender, Token token);

    ~nest-sender();

private:
    optional<wrapped-sender-from<Token, Sender>> sender_;
    association-from<Token> token;
};

template <sender Sender, async_scope_token Token>
auto nest(Sender&& snd, Token token)
    noexcept(is_nothrow_constructible_v<nest-sender<Sender, Token>, Sender, Token>)
    -> nest-sender<Sender, Token>;

template <sender Sender, async_scope_token Token, class Env = empty_env>
void spawn(Sender&& snd, Token token, Env env = {})
    requires sender_to<decltype(token.wrap(forward<Sender>(snd))),
        spawn-receiver<Env>>;

template <sender Sender, async_scope_token Token, class Env = empty_env>
future-sender-t<Sender, Env> spawn_future(Sender&& snd, Token token, Env env = {});

struct simple_counting_scope {
    simple_counting_scope() noexcept;
    ~simple_counting_scope();

    // simple_counting_scope is immovable and uncopyable
    simple_counting_scope(const simple_counting_scope&) = delete;
    simple_counting_scope(simple_counting_scope&&) = delete;
    simple_counting_scope& operator=(const simple_counting_scope&) = delete;
    simple_counting_scope& operator=(simple_counting_scope&&) = delete;

    struct token;

    struct assoc {
        assoc() noexcept = default;

        assoc(const assoc&) noexcept;

        assoc(assoc&&) noexcept;

        ~assoc();

        assoc& operator=(assoc) noexcept;

        explicit operator bool() const noexcept;

private:
        friend token;

        explicit assoc(simple_counting_scope*) noexcept; // exposition-only

        simple_counting_scope* scope_{}; // exposition-only
    };

    struct token {
        template <sender Sender>
        Sender&& wrap(Sender&& snd) const noexcept;

        assoc try_associate() const;

private:
        friend simple_counting_scope;

        explicit token(simple_counting_scope* s) noexcept; // exposition-only
    };
};

```

```

        simple_counting_scope* scope_; // exposition-only
    };

    token get_token() noexcept;

    void close() noexcept;

    struct join_sender; // exposition-only

    join_sender join() noexcept;
};

struct counting_scope {
    counting_scope() noexcept;
    ~counting_scope();

    // counting_scope is immovable and uncopyable
    counting_scope(const counting_scope&) = delete;
    counting_scope(counting_scope&&) = delete;
    counting_scope& operator=(const counting_scope&) = delete;
    counting_scope& operator=(counting_scope&&) = delete;

    template <sender Sender>
    struct wrapper_sender; // exposition-only

    struct token {
        template <sender Sender>
        wrapper_sender<Sender> wrap(Sender&& snd) const;

        async_scope_association auto try_associate() const;

    private:
        friend counting_scope;

        explicit token(counting_scope* s) noexcept; // exposition-only

        counting_scope* scope_; // exposition-only
    };

    token get_token() noexcept;

    void close() noexcept;

    void request_stop() noexcept;

    struct join_sender; // exposition-only

    join_sender join() noexcept;
};

```

§ 5.2 execution::async_scope_association

```

template <class Assoc>
concept async_scope_association =
    semiregular<Assoc> &&
    requires(const Assoc& assoc) {
        { static_cast<bool>(assoc) } noexcept;
    };

```

An async scope association is an RAII handle type that represents a possible association between a sender and an async scope. If the scope association contextually converts to true then the object is “engaged” and represents an association; otherwise, the object is “disengaged” and represents the lack of an association. Async scope associations are copyable but, when copying an engaged association, the resulting copy may be disengaged because the underlying async scope may decline to create a new association.

§ 5.3 execution::async_scope_token

```

template <class Token>
concept async_scope_token =
    copyable<Token> &&
    requires(Token token) {
        { token.try_associate() } -> async_scope_association;
    };

```

An async scope token is a non-owning handle to an async scope. The try_associate() method on a token attempts to create a new association with the scope; try_associate() returns an engaged association when the association is successful, and it may either return a disengaged association or throw an exception to indicate failure. Returning a disengaged association will generally lead to algorithms that

operate on tokens behaving as if provided a sender that completes immediately with `set_stopped()`, leading to rejected work being discarded as a “no-op”. Throwing an exception will generally lead to that exception escaping from the calling algorithm.

Tokens also have a `wrap()` method that takes and returns a sender. The `wrap()` method gives the token an opportunity to modify the input sender’s behaviour in a scope-specific way. The proposed `counting_scope` uses this opportunity to associate the input sender with a stop token that the scope can use to request stop on all outstanding operations associated within the scope.

In order to provide the Strong Exception Guarantee, the algorithms proposed in this paper invoke `token.wrap(snd)` before invoking `token.try_associate()`. Other algorithms written in terms of `async_scope_token` should do the same.

The following sketch implementation of *nest-sender* illustrates how the methods on an async scope token interact:

```
template <sender Sender, async_scope_token Token>
struct nest-sender {
    nest-sender(Sender&& s, Token t)
        : sender_(t.wrap(forward<Sender>(s))) {
        assoc_ = t.try_associate();
        if (!assoc_) {
            sender_.reset(); // assume no_throw destructor
        }
    }

    nest-sender(const nest-sender& other)
        requires copy_constructible<wrapped-sender-from<Token, Sender>>
        : assoc_(t.try_associate()) {
        if (assoc_) {
            sender_ = other.sender_;
        }
    }

    nest-sender(nest-sender&& other) noexcept = default;

    ~nest-sender() = default;

    // ... implement the sender concept in terms of Sender and sender_
private:
    association-from<Token> assoc_;
    optional<wrapped-sender-from<Token, Sender>> sender_;
};
```

An async scope token behaves like a reference-to-async-scope; tokens are no-throw copyable and movable, and it is undefined behaviour to invoke any methods on a token that has outlived its scope.

§ 5.4 execution::nest

```
template <sender Sender, async_scope_token Token>
auto nest(Sender&& snd, Token token)
    noexcept(is_nothrow_constructible_v<nest-sender<Sender, Token>, Sender, Token>)
    -> nest-sender<Sender, Token>;
```

When successful, `nest()` creates an association with the given token’s scope and returns an “associated” sender that behaves the same as its input sender, with the following additional effects:

- the association ends when the returned sender is destroyed or, if it is connected, when the resulting operation state is destroyed; and
- whatever effects are added by the token’s `wrap()` method.

When unsuccessful, `nest()` will either return an “unassociated” sender or it will allow any thrown exceptions to escape.

When `nest()` returns an associated sender:

- connecting and starting the associated sender connects and starts the given sender; and
- the associated sender has exactly the same completions as the input sender.

When `nest()` returns an unassociated sender:

- the input sender is discarded and will never be connected or started; and
- the unassociated sender will only complete with `set_stopped()`.

`nest()` simply constructs and returns a *nest-sender*. Given an `async_scope_token`, `token`, and a sender, `snd`, the *nest-sender* constructor performs the following operations in the following order:

1. store the result of `token.wrap(snd)` in a member variable

2. store the result of `token.try_associate()` in a member variable

- a. if the resulting association is disengaged then destroy the previously stored result of `token.wrap(snd)`; the *nest-sender* under construction is an unassociated sender.
- b. otherwise, the *nest-sender* under construction is an associated sender.

Any exceptions thrown during the evaluation of the constructor are allowed to escape; nevertheless, `nest()` provides the Strong Exception Guarantee.

An associated *nest-sender* has many properties of an RAII handle:

- constructing an instance acquires a “resource” (the association with the scope)
- destructing an instance releases the same resource
- moving an instance into another transfers ownership of the resource from the source to the destination
- etc.

Copying a *nest-sender* is possible if the sender it is wrapping is copyable but the copying process is a bit unusual because of the `async_scope_association` it contains. If the sender, `snd`, provided to `nest()` is copyable then the resulting *nest-sender* is also copyable, with the following rules:

- copying an unassociated *nest-sender* invariably produces a new unassociated *nest-sender*; and
- copying an associated *nest-sender* proceeds as follows:
 1. copy the association from the source into the destination *nest-sender*
 - if the copied association is engaged then copy the wrapped sender from the source into the destination *nest-sender*; the destination is associated
 - otherwise, the destination is unassociated

When *nest-sender* has a copy constructor, it provides the Strong Exception Guarantee.

When connecting an unassociated *nest-sender*, the resulting *operation-state* completes immediately with `set_stopped()` when started.

When connecting an associated *nest-sender*, there are four possible outcomes:

1. the *nest-sender* is rvalue connected, which infallibly moves the sender’s association from the sender to the *operation-state*
2. the *nest-sender* is lvalue connected, in which case the sender’s association must be copied into the *operation-state*, which may:
 - a. succeed by creating a new engaged association for the *operation-state*;
 - b. fail by creating a new disengaged association for the *operation-state*, in which case the new *operation-state* behaves as if it were constructed from an unassociated *nest-sender*; or
 - c. fail by throwing an exception, in which case the exception escapes from the call to connect.

An *operation-state* with its own association must invoke the association’s destructor as the last step of the *operation-state*’s destructor.

Note: the timing of when an associated *operation-state* ends its association with the scope is chosen to avoid exposing user code to dangling references. Scopes are expected to serve as mechanisms for signaling when it is safe to destroy shared resources being protected by the scope. Ending any given association with a scope may lead to that scope signaling that the protected resources can be destroyed so a *nest-sender*’s *operation-state* must not permit that signal to be sent until the *operation-state* is definitely finished accessing the shared resources, which is at the end of the *operation-state*’s destructor.

A call to `nest()` does not start the given sender and is not expected to incur allocations.

Regardless of whether the returned sender is associated or unassociated, it is multi-shot if the input sender is multi-shot and single-shot otherwise.

§ 5.5 `execution::spawn`

```
namespace { // exposition-only
template <class Env>
struct spawn-env; // exposition-only

template <class Env>
struct spawn-receiver { // exposition-only
    void set_value() noexcept;
```

```

    void set_stopped() noexcept;

    const spawn-env<Env>& get_env() const noexcept;
};

}

template <sender Sender, async_scope_token Token, class Env = empty_env>
void spawn(Sender&& snd, Token token, Env env = {})
    requires sender_to<decltype(token.wrap(forward<Sender>(snd)))>,
             spawn-receiver<End>>;

```

Attempts to associate the given sender with the given scope token's scope. On success, the given sender is eagerly started. On failure, either the sender is discarded and no further work happens or `spawn()` throws.

Starting the given sender without waiting for it to finish requires a dynamic allocation of the sender's *operation-state*. The following algorithm determines which *Allocator* to use for this allocation:

- If `get_allocator(env)` is valid and returns an *Allocator* then choose that *Allocator*.
- Otherwise, if `get_allocator(get_env(snd))` is valid and returns an *Allocator* then choose that *Allocator*.
- Otherwise, choose `std::allocator<>`.

`spawn()` proceeds with the following steps in the following order:

1. the type of the object to dynamically allocate is computed, say `op_t`; `op_t` contains
 - an *operation-state*;
 - an allocator of the chosen type; and
 - an association of type `decltype(token.try_associate())`.
2. an `op_t` is dynamically allocated by the *Allocator* chosen as described above
3. the fields of the `op_t` are initialized in the following order:
 - a. the *operation-state* within the allocated `op_t` is initialized with the result of `connect(token.wrap(forward<Sender>(sender)), spawn-receiver{...})`;
 - b. the allocator is initialized with a copy of the allocator used to allocate the `op_t`; and
 - c. the association is initialized with the result of `token.try_associate()`.
4. if the association in the `op_t` is engaged then the *operation-state* is started; otherwise, the `op_t` is destroyed and deallocated.

Any exceptions thrown during the execution of `spawn()` are allowed to escape; nevertheless, `spawn()` provides the Strong Exception Guarantee.

Upon completion of the *operation-state*, the *spawn-receiver* performs the following steps:

1. move the allocator and association from the `op_t` into local variables;
2. destroy the *operation-state*;
3. use the local copy of the allocator to deallocate the `op_t`;
4. destroy the local copy of the allocator; and
5. destroy the local copy of the association.

Performing step 5 last ensures that all possible references to resources protected by the scope, including possibly the allocator, are no longer in use before dissociating from the scope.

A *spawn-receiver*, `sr`, responds to `get_env(sr)` with an instance of a *spawn-env*<Env>, `senv`. The result of `get_allocator(senv)` is a copy of the *Allocator* used to allocate the *operation-state*. For all other queries, `Q`, the result of `Q(senv)` is `Q(env)`.

This is similar to `start_detached()` from [P2300R7], but the scope may observe and participate in the lifetime of the work described by the sender. The `simple_counting_scope` and `counting_scope` described in this paper use this opportunity to keep a count of spawned senders that haven't finished, and to prevent new senders from being spawned once the scope has been closed.

The given sender must complete with `set_value()` or `set_stopped()` and may not complete with an error; the user must explicitly handle the errors that might appear as part of the *sender-expression* passed to `spawn()`.

User expectations will be that `spawn()` is asynchronous and so, to uphold the principle of least surprise, `spawn()` should only be given non-blocking senders. Using `spawn()` with a sender generated by `on(sched, blocking-sender)` is a very useful pattern in this context.

NOTE: A query for non-blocking start will allow `spawn()` to be constrained to require non-blocking start.

Usage example:

```
...  
for (int i = 0; i < 100; i++)  
    spawn(on(sched, some_work(i)), scope.get_token());
```

§ 5.6 execution::spawn_future

```
namespace { // exposition-only  
  
template <class Env>  
struct future-env; // exposition-only  
  
template <valid-completion-signatures Sigs>  
struct future-sender; // exposition-only  
  
template <sender Sender, class Env>  
using future-sender-t = // exposition-only  
    future-sender<completion_signatures_of_t<Sender, future-env<Env>>>;  
  
}  
  
template <sender Sender, async_scope_token Token, class Env = empty_env>  
future-sender-t<Sender, Env> spawn_future(Sender&& snd, Token token, Env env = {});
```

Attempts to associate the given sender with the given scope token's scope. On success, the given sender is eagerly started and `spawn_future` returns a *future-sender* that provides access to the result of the given sender. On failure, either `spawn_future` returns a *future-sender* that unconditionally completes with `set_stopped()` or it throws.

Similar to `spawn()`, starting the given sender involves a dynamic allocation of some state. `spawn_future()` chooses an *Allocator* for this allocation in the same way `spawn()` does: use the result of `get_allocator(env)` if that is a valid expression, otherwise use the result of `get_allocator(get_env(snd))` if that is a valid expression, otherwise use a `std::allocator<>`.

Compared to `spawn()`, the dynamically allocated state is more complicated because it must contain storage for the result of the given sender, however it eventually completes, and synchronization facilities for resolving the race between the given sender's production of its result and the returned sender's consumption or abandonment of that result.

Unlike `spawn()`, `spawn_future()` returns a *future-sender* rather than `void`. The returned sender, `fs`, is a handle to the spawned work that can be used to consume or abandon the result of that work. The completion signatures of `fs` include `set_stopped()` and all the completion signatures of the spawned sender. When `fs` is connected and started, it waits for the spawned sender to complete and then completes itself with the spawned sender's result.

The receiver, `fr`, that is connected to the given sender responds to `get_env(fr)` with an instance of *future-env<Env>*, `fenv`. The result of `get_allocator(fenv)` is a copy of the *Allocator* used to allocate the dynamically allocated state. The result of `get_stop_token(fenv)` is a stop token that will be "triggered" (i.e. signal that stop is requested) when:

- the returned *future-sender* is dropped;
- the returned *future-sender* receives a stop request; or
- the stop token returned from `get_stop_token(env)` is triggered if `get_stop_token(env)` is a valid expression.

For all other queries, `Q`, the result of `Q(fenv)` is `Q(env)`.

`spawn_future()` proceeds with the following steps in the following order:

1. storage for the spawned sender's state is dynamically allocated by the *Allocator* chosen as described above
2. the state for the spawned sender is constructed in the allocated storage
 - a subset of this state is an *operation-state* created by connecting the result of `token.wrap(forward<Sender>(sender))` with a receiver
 - the last field to be initialized in the dynamically allocated state is an async scope association that is initialized with the result of `token.try_associate()`
 - if the resulting association is engaged then
 - the *operation-state* within the allocated state is started; and
 - a *future-sender* is returned that, when connected and started, will complete with the result of the eagerly-started work
 - otherwise

- the dynamically-allocated state is destroyed and deallocated; and
- a *future-sender* is returned that will complete with `set_stopped()`

Any exceptions thrown during the execution of `spawn_future()` are allowed to escape; nevertheless, `spawn_future()` provides the Strong Exception Guarantee.

Given a *future-sender*, `fs`, if `fs` is destroyed without being connected, or if it is connected and the resulting *operation-state*, `fsop`, is destroyed without being started, then the eagerly-started work is “abandoned”.

Abandoning the eagerly-started work means:

- a stop request is sent to the running *operation-state*;
- any result produced by the running *operation-state* is discarded when the operation completes; and
- after the operation completes, the dynamically-allocated state is “cleaned up”.

Cleaning up the dynamically-allocated state means doing the following, in order:

1. the allocator and association in the state are moved into local variables;
2. the state is destroyed;
3. the dynamic allocation is deallocated with the local copy of the allocator;
4. the local copy of the allocator is destroyed; and
5. the local copy of the association is destroyed.

When `fsop` is started, if `fsop` receives a stop request from its receiver before the eagerly-started work has completed then an attempt is made to abandon the eagerly-started work. Note that it’s possible for the eagerly-started work to complete while `fsop` is requesting stop; once the stop request has been delivered, either `fsop` completes with the result of the eagerly-started work if it’s ready, or it completes with `set_stopped()` without waiting for the eagerly-started work to complete.

When `fsop` is started and does not receive a stop request from its receiver, `fsop` completes after the eagerly-started work completes with the same completion. Once `fsop` completes, it cleans up the dynamically-allocated state.

`spawn_future` is similar to `ensure_started()` from [P2300R7], but the scope may observe and participate in the lifetime of the work described by the sender. The `simple_counting_scope` and `counting_scope` described in this paper use this opportunity to keep a count of given senders that haven’t finished, and to prevent new senders from being started once the scope has been closed.

Unlike `spawn()`, the sender given to `spawn_future()` is not constrained on a given shape. It may send different types of values, and it can complete with errors.

Usage example:

```
...
sender auto snd = spawn_future(on(sched, key_work()), token) | then(continue_fun);
for (int i = 0; i < 10; i++)
    spawn(on(sched, other_work(i)), token);
return when_all(scope.join(), std::move(snd));
```

§ 5.7 execution::simple_counting_scope

```
struct simple_counting_scope {
    simple_counting_scope() noexcept;
    ~simple_counting_scope();

    // simple_counting_scope is immovable and uncopyable
    simple_counting_scope(const simple_counting_scope&) = delete;
    simple_counting_scope(simple_counting_scope&&) = delete;
    simple_counting_scope& operator=(const simple_counting_scope&) = delete;
    simple_counting_scope& operator=(simple_counting_scope&&) = delete;

    struct token;

    struct assoc {
        assoc() noexcept = default;

        assoc(const assoc&) noexcept;

        assoc(assoc&&) noexcept;

        ~assoc();
```

```

    assoc& operator=(assoc) noexcept;

    explicit operator bool() const noexcept;

private:
    friend token;

    explicit assoc(simple_counting_scope*) noexcept; // exposition-only

    simple_counting_scope* scope_{}; // exposition-only
};

struct token {
    template <sender Sender>
    Sender&& wrap(Sender&& snd) const noexcept;

    assoc try_associate() const;

private:
    friend simple_counting_scope;

    explicit token(simple_counting_scope* s) noexcept; // exposition-only

    simple_counting_scope* scope_; // exposition-only
};

token get_token() noexcept;

void close() noexcept;

struct join_sender; // exposition-only

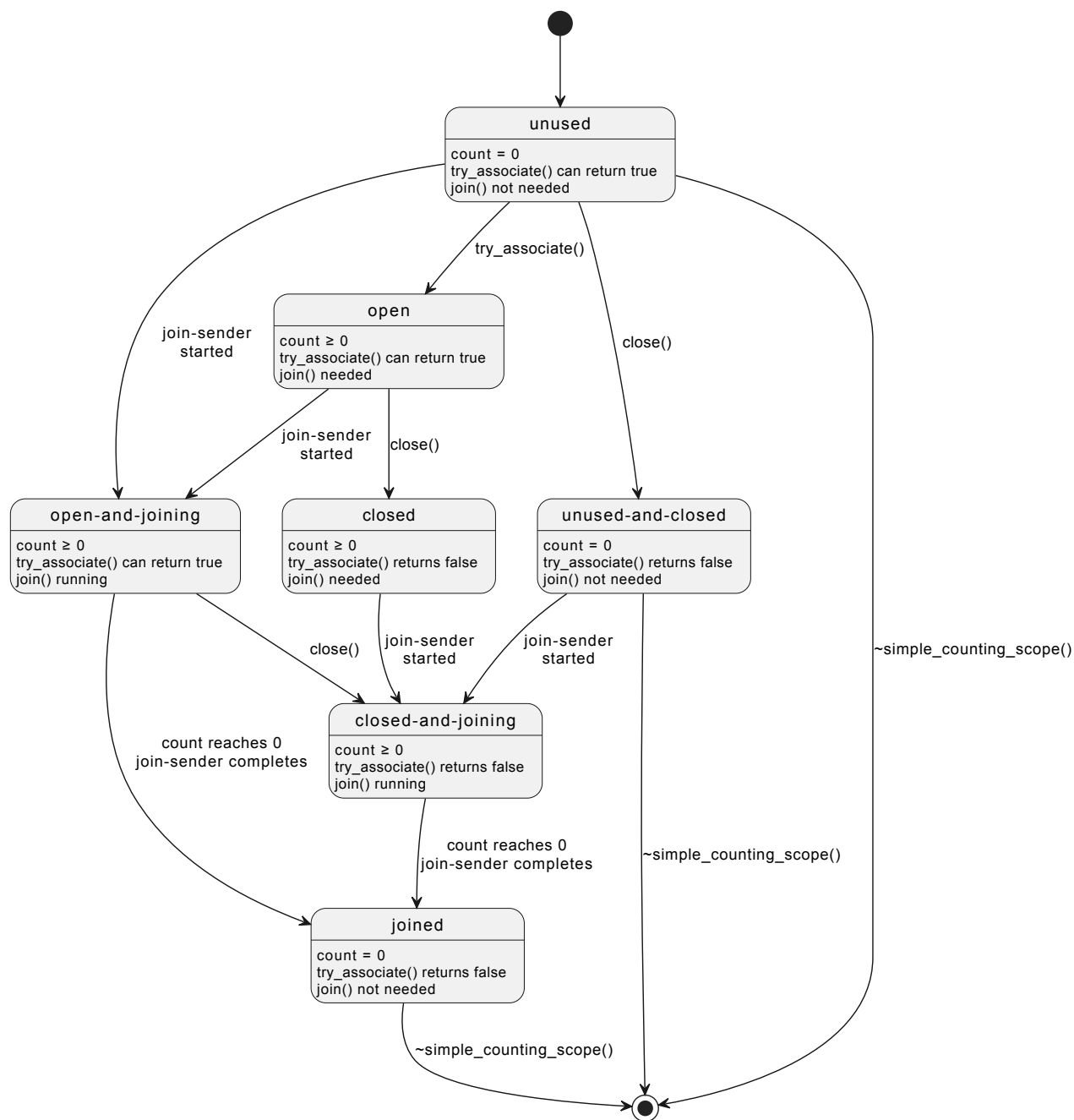
join_sender join() noexcept;
};

```

A `simple_counting_scope` maintains a count of outstanding operations and goes through several states during its lifetime:

- unused
- open
- closed
- open-and-joining
- closed-and-joining
- unused-and-closed
- joined

The following diagram illustrates the `simple_counting_scope`'s state machine:



Note: a scope is “open” if its current state is unused, open, or open-and-joining; a scope is “closed” if its current state is closed, unused-and-closed, closed-and-joining, or joined.

Instances start in the unused state after being constructed. This is the only time the scope’s state can be set to unused. When the `simple_counting_scope` destructor starts, the scope must be in the unused, unused-and-closed, or joined state; otherwise, the destructor invokes `std::terminate()`. Permitting destruction when the scope is in the unused or unused-and-closed state ensures that instances of `simple_counting_scope` can be used safely as data-members while preserving structured functionality.

Connecting and starting a *join-sender* returned from `join()` moves the scope to either the open-and-joining or closed-and-joining state. Merely calling `join()` or connecting the *join-sender* does not change the scope’s state—the *operation-state* must be started to effect the state change. A started *join-sender* completes when the scope’s count of outstanding operations reaches zero, at which point the scope transitions to the joined state.

Calling `close()` on a `simple_counting_scope` moves the scope to the closed, unused-and-closed, or closed-and-joining state, and causes all future calls to `try_associate()` to return false.

Associating work with a `simple_counting_scope` can be done through `simple_counting_scope`’s token. `simple_counting_scope`’s token provides 2 methods: `wrap(Sender&& s)`, and `try_associate()`.

— `wrap(Sender&&s)` takes in a sender and returns it unmodified.

— `try_associate()` attempts to create a new association with the `simple_counting_scope` and will return an engaged association when successful, or a disengaged association otherwise. The requirements for `try_associate()`'s success are outlined below:

1. While a scope is in the unused, open, or open-and-joining state, calls to `token.try_associate()` succeeds by incrementing the scope's count of outstanding operations before returning an engaged association.
2. While a scope is in the closed, unused-and-closed, closed-and-joining, or joined state, calls to `token.try_associate()` will return a disengaged association and *will not* increment the scope's count of outstanding operations.

When a token's `try_associate()` returns an engaged association, the destructor of the resulting association will undo the association by decrementing the scope's count of outstanding operations.

— When a scope is in the open-and-joining or closed-and-joining state and an association's destructor undoes the final scope association, the scope moves to the joined state and the outstanding *join-sender* completes.

The state transitions of a `simple_counting_scope` mean that it can be used to protect asynchronous work from use-after-free errors. Given a resource, `res`, and a `simple_counting_scope`, `scope`, obeying the following policy is enough to ensure that there are no attempts to use `res` after its lifetime ends:

- all senders that refer to `res` are associated with `scope`; and
- `scope` is destroyed (and therefore in the joined, unused, or unused-and-closed state) before `res` is destroyed.

It is safe to destroy a scope in the unused or unused-and-closed state because there can't be any work referring to the resources protected by the scope.

A `simple_counting_scope` is uncopyable and immovable so its copy and move operators are explicitly deleted. `simple_counting_scope` could be made movable but it would cost an allocation so this is not proposed.

§ 5.7.1 `simple_counting_scope::simple_counting_scope`

```
simple_counting_scope() noexcept;
```

Initializes a `simple_counting_scope` in the unused state with the count of outstanding operations set to zero.

§ 5.7.2 `simple_counting_scope::~simple_counting_scope`

```
~simple_counting_scope();
```

Checks that the `simple_counting_scope` is in the joined, unused, or unused-and-closed state and invokes `std::terminate()` if not.

§ 5.7.3 `simple_counting_scope::get_token`

```
simple_counting_scope::token get_token() noexcept;
```

Returns a `simple_counting_scope::token` referring to the current scope, as if by invoking `token{this}`.

§ 5.7.4 `simple_counting_scope::close`

```
void close() noexcept;
```

Moves the scope to the closed, unused-and-closed, or closed-and-joining state. After a call to `close()`, all future calls to `try_associate()` return false.

§ 5.7.5 `simple_counting_scope::join`

```
struct join_sender; // exposition-only
join_sender join() noexcept;
```

Returns a *join-sender*. When the *join-sender* is connected to a receiver, `r`, it produces an *operation-state*, `o`. When `o` is started, the scope moves to either the open-and-joining or closed-and-joining state. `o` completes with `set_value()` when the scope moves to the joined state, which happens when the scope's count of outstanding senders drops to zero. `o` may complete synchronously if it happens to observe that the count of outstanding senders is already zero when started; otherwise, `o` completes on the execution context associated with the scheduler in its receiver's environment by asking its receiver, `r`, for a scheduler, `sch`, with `get_scheduler(get_env(r))` and then starting the sender returned from `schedule(sch)`. This requirement to complete on the receiver's scheduler restricts which receivers a *join-sender* may be connected to in exchange for determinism; the alternative would have the *join-sender* completing on the execution context of whichever nested operation happens to be the last one to complete.

§ 5.7.6 `simple_counting_scope::assoc::assoc`

```
assoc() noexcept = default;
```



```
explicit assoc(simple_counting_scope*) noexcept; // exposition-only
assoc(const assoc&) noexcept;
assoc(assoc&&) noexcept;
```

The default `assoc` constructor produces a disengaged association.

The private, exposition-only constructor accepting a `simple_counting_scope*` either:

- constructs a disengaged association if the given pointer is `nullptr`; or
- constructs an engaged association associated with the given scope.

The copy constructor either:

- infallibly copies a disengaged association; or
- attempts to create a new association as if by invoking `get_token().try_associate()` on the source association's scope.

The move constructor either:

- infallibly produces a disengaged association from a disengaged association; or
- infallibly transfers the association from an engaged association to the new object, leaving the source disengaged.

§ 5.7.7 `simple_counting_scope::assoc::~~assoc`

```
~assoc();
```

The `assoc` destructor either:

- does nothing if the association is disengaged; or
- decrements the associated scope's count of outstanding operations and, when the scope is in the open-and-joining or closed-and-joining state, moves the scope to the joined state and signals the outstanding *join-sender* to complete.

§ 5.7.8 `simple_counting_scope::assoc::operator=`

```
assoc& operator=(assoc) noexcept;
```

The assignment operator behaves as if it is implemented as follows:

```
assoc& operator=(assoc rhs) noexcept
{
    swap(scope_, rhs.scope_);
    return *this;
}
```

where `scope_` is a private member of type `simple_counting_scope*` that points to the association's associated scope.

§ 5.7.9 `simple_counting_scope::assoc::operator bool`

```
explicit operator bool() const noexcept;
```

Returns `true` when the association is engaged and `false` when it is disengaged.

§ 5.7.10 `simple_counting_scope::token::wrap`

```
template <sender Sender>
Sender&& wrap(Sender&& s) const noexcept;
```

Returns the argument unmodified.

§ 5.7.11 `simple_counting_scope::token::try_associate`

```
assoc try_associate() const;
```

The following atomic state change is attempted on the token's scope:

- increment the scope's count of outstanding operations; and
- move the scope to the open state if it was in the unused state.

The atomic state change succeeds and the method returns an engaged `assoc` if the scope is observed to be in the unused, open, or open-and-joining state; otherwise the scope's state is left unchanged and the method returns a disengaged `assoc`.

§ 5.8 `execution::counting_scope`

```

struct counting_scope {
    counting_scope() noexcept;
    ~counting_scope();

    // counting_scope is immovable and uncopyable
    counting_scope(const counting_scope&) = delete;
    counting_scope(counting_scope&&) = delete;
    counting_scope& operator=(const counting_scope&) = delete;
    counting_scope& operator=(counting_scope&&) = delete;

    template <sender Sender>
    struct wrapper_sender; // exposition-only

    struct token {
        template <sender Sender>
        wrapper_sender<Sender> wrap(Sender&& snd);

        async_scope_association auto try_associate() const;

    private:
        friend counting_scope;

        explicit token(counting_scope* s) noexcept; // exposition-only

        counting_scope* scope; // exposition-only
    };

    token get_token() noexcept;

    void close() noexcept;

    void request_stop() noexcept;

    struct join_sender; // exposition-only

    join_sender join() noexcept;
};

```

A `counting_scope` augments a `simple_counting_scope` with a stop source and gives to each of its associated `wrapper_senders` a stop token from its stop source. This extension of `simple_counting_scope` allows a `counting_scope` to request stop on all of its outstanding operations by requesting stop on its stop source.

Assuming an exposition-only `stop_when(sender auto&&, stoppable_token auto)` (explained below), `counting_scope` behaves as if it were implemented like so:

```

struct counting_scope {
    struct token {
        template <sender S>
        sender auto wrap(S&& snd) const
            noexcept(std::is_nothrow_constructible_v<std::remove_cvref_t<S>, S>) {
            return stop_when(std::forward<S>(snd), scope_>source_.get_token());
        }

        async_scope_association auto try_associate() const {
            return scope_>scope_.get_token().try_associate();
        }

    private:
        friend counting_scope;

        explicit token(counting_scope* scope) noexcept
            : scope_(scope) {}

        counting_scope* scope_;
    };

    token get_token() noexcept { return token{this}; }

    void close() noexcept { return scope_.close(); }

    void request_stop() noexcept { source_.request_stop(); }

    sender auto join() noexcept { return scope_.join(); }

private:
    simple_counting_scope scope_;
    inplace_stop_source source_;
};

```

`stop_when(sender auto&& snd, stoppable_token auto token)` is an exposition-only sender algorithm that maps its input sender, `snd`, to an output sender, `osnd`, such that, when `osnd` is connected to a receiver, `r`, the resulting *operation-state* behaves the same as connecting the original sender, `snd`, to `r`, except that `snd` will receive a stop request when either the token returned from `get_stop_token(r)` receives a stop request or when `token` receives a stop request.

Other than the use of `stop_when()` in `counting_scope::token::wrap()` and the addition of `request_stop()` to the interface, `counting_scope` has the same behavior and lifecycle as `simple_counting_scope`.

§ 5.8.1 `counting_scope::counting_scope`

```
counting_scope() noexcept;
```

Initializes a `counting_scope` in the unused state with the count of outstanding operations set to zero.

§ 5.8.2 `counting_scope::~~counting_scope`

```
~counting_scope();
```

Checks that the `counting_scope` is in the joined, unused, or unused-and-closed state and invokes `std::terminate()` if not.

§ 5.8.3 `counting_scope::get_token`

```
counting_scope::token get_token() noexcept;
```

Returns a `counting_scope::token` referring to the current scope, as if by invoking `token{this}`.

§ 5.8.4 `counting_scope::close`

```
void close() noexcept;
```

Moves the scope to the closed, unused-and-closed, or closed-and-joining state. After a call to `close()`, all future calls to `nest()` that return normally return unassociated senders.

§ 5.8.5 `counting_scope::request_stop`

```
void request_stop() noexcept;
```

Requests stop on the scope's internal stop source. Since all senders nested within the scope have been given stop tokens from this internal stop source, the effect is to send stop requests to all outstanding (and future) nested operations.

§ 5.8.6 `counting_scope::join`

```
struct join_sender; // exposition-only  
join_sender join() noexcept;
```

Returns a `join_sender` that behaves the same as the result of `simple_counting_scope::join()`. Connecting and starting the `join_sender` moves the scope to the open-and-joining or closed-and-joining state; the `join_sender` completes when the scope's count of outstanding operations drops to zero, at which point the scope moves to the joined state.

§ 5.8.7 `counting_scope::token::wrap`

```
template <sender S>  
struct wrapper_sender; // exposition-only  
  
template <sender Sender>  
wrapper_sender<Sender> wrap(Sender&& snd);
```

Returns a `wrapper_sender<Sender>`, `osnd`, that behaves in all ways the same as the input sender, `snd`, except that, when `osnd` is connected to a receiver, the resulting *operation-state* receives stop requests from *both* the connected receiver *and* the stop source in the token's `counting_scope`.

§ 5.8.8 `counting_scope::token::try_associate`

```
async_scope_association auto try_associate() const;
```

Returns an `async_scope_association` that is engaged if the token's scope is open, and disengaged if it's closed. `try_associate()` behaves as if its `counting_scope` owns a `simple_counting_scope`, `scope`, and the result is equivalent to the result of invoking `scope.get_token().try_associate()`.

§ 5.9 When to use `counting_scope` vs [P3296R2]'s `let_async_scope`

Although `counting_scope` and `let_async_scope` have overlapping use-cases, we specifically designed the two facilities to address separate problems. In short, `counting_scope` is best used in an unstructured context and `let_async_scope` is best used in a structured context.

We define “unstructured context” as:

- a place where using `sync_wait` would be inappropriate,
- and you can’t “solve by induction” (i.e you’re not in an `async` context where you can start the sender by “awaiting” it)

`counting_scope` should be used when you have a sender you want to start in an unstructured context. In this case, `spawn(sender, scope.get_token())` would be the preferred way of starting asynchronous work. `scope.join()` needs to be called before the owning object’s destruction in order to ensure that the object’s lifetime lives at least until all asynchronous work completes. Note that exception safety needs to be handled explicitly in the use of `counting_scope`.

`let_async_scope` returns a sender, and therefore can only be started in one of 3 ways:

1. `sync_wait`
2. `spawn` on a `counting_scope`
3. `co_await`

`let_async_scope` will manage the scope for you, ensuring that the managed scope is always joined before `let_async_scope` completes. The algorithm frees the user from having to manage the coupling between the lifetimes of the managed scope and the resource(s) it protects with the limitation that the nested work must be fully structured. This behavior is a feature, since the scope being managed by `let_async_scope` is intended to live only until the sender completes. This also means that `let_async_scope` will be exception safe by default.

§ 6 Design considerations

§ 6.1 Shape of the given sender

§ 6.1.1 Constraints on `set_value()`

It makes sense for `spawn_future()` and `nest()` to accept senders with any type of completion signatures. The caller gets back a sender that can be chained with other senders, and it doesn’t make sense to restrict the shape of this sender.

The same reasoning doesn’t necessarily follow for `spawn()` as it returns `void` and the result of the spawned sender is dropped. There are two main alternatives:

- do not constrain the shape of the input sender (i.e., dropping the results of the computation)
- constrain the shape of the input sender

The current proposal goes with the second alternative. The main reason is to make it more difficult and explicit to silently drop results. The caller can always transform the input sender before passing it to `spawn()` to drop the values manually.

Chosen: `spawn()` accepts only senders that advertise `set_value()` (without any parameters) in the completion signatures.

§ 6.1.2 Handling errors in `spawn()`

The current proposal does not accept senders that can complete with error given to `spawn()`. This will prevent accidental error scenarios that will terminate the application. The user must deal with all possible errors before passing the sender to `spawn()`. i.e., error handling must be explicit.

Another alternative considered was to call `std::terminate()` when the sender completes with error.

Another alternative is to silently drop the errors when receiving them. This is considered bad practice, as it will often lead to first spotting bugs in production.

Chosen: `spawn()` accepts only senders that do not call `set_error()`. Explicit error handling is preferred over stopping the application, and over silently ignoring the error.

§ 6.1.3 Handling stop signals in `spawn()`

Similar to the error case, we have the alternative of allowing or forbidding `set_stopped()` as a completion signal. Because the goal of `counting_scope` is to track the lifetime of the work started through it, it shouldn’t matter whether that the work completed with success or by being stopped. As it is assumed that sending the stop signal is the result of an explicit choice, it makes sense to allow senders that can terminate with `set_stopped()`.

The alternative would require transforming the sender before passing it to `spawn`, something like `spawn(std::move(snd) | let_stopped(just), s.get_token())`. This is considered boilerplate and not helpful, as the stopped scenarios should be implicit, and not require handling.

Chosen: `spawn()` accepts senders that complete with `set_stopped()`.

§ 6.1.4 No shape restrictions for the senders passed to `spawn_future()` and `nest()`

Similarly to `spawn()`, we can constrain `spawn_future()` and `nest()` to accept only a limited set of senders. But, because we can attach continuations for these senders, we would be limiting the functionality that can be expressed. For example, the continuation can handle different types of values and errors.

Chosen: `spawn_future()` and `nest()` accept senders with any completion signatures.

§ 6.2 P2300's `start_detached()`

The `spawn()` algorithm in this paper can be used as a replacement for `start_detached` proposed in [P2300R7]. Essentially it does the same thing, but it also provides the given scope the opportunity to apply its bookkeeping policy to the given sender, which, in the case of `counting_scope`, ensures the program can wait for spawned work to complete before destroying any resources references by that work.

§ 6.3 P2300's `ensure_started()`

The `spawn_future()` algorithm in this paper can be used as a replacement for `ensure_started` proposed in [P2300R7]. Essentially it does the same thing, but it also provides the given scope the opportunity to apply its bookkeeping policy to the given sender, which, in the case of `counting_scope`, ensures the program can wait for spawned work to complete before destroying any resources references by that work.

§ 6.4 Supporting the pipe operator

This paper doesn't support the pipe operator to be used in conjunction with `spawn()` and `spawn_future()`. One might think that it is useful to write code like the following:

```
std::move(snd1) | spawn(s); // returns void
sender auto snd3 = std::move(snd2) | spawn_future(s) | then(...);
```

In [P2300R7] sender consumers do not have support for the pipe operator. As `spawn()` works similarly to `start_detached()` from [P2300R7], which is a sender consumer, if we follow the same rationale, it makes sense not to support the pipe operator for `spawn()`.

On the other hand, `spawn_future()` is not a sender consumer, thus we might have considered adding pipe operator to it.

On the third hand, Unifex supports the pipe operator for both of its equivalent algorithms (`unifex::spawn_detached()` and `unifex::spawn_future()`) and Unifex users have not been confused by this choice.

To keep consistency with `spawn()` this paper doesn't support pipe operator for `spawn_future()`.

§ 7 Naming

As is often true, naming is a difficult task. We feel more confident about having arrived at a reasonably good naming *scheme* than good *names*:

- There is some consensus that the default standard “scope” should be the one this paper calls `counting_scope` because it provides all of the obviously-useful features of a scope, while `simple_counting_scope` is the more spare type that only provides scoping facilities. Therefore, `counting_scope` should get the “nice” name, while `simple_counting_scope` should get a more cumbersome name that conveys fewer features in exchange for a smaller object size and fewer atomic operations.
- Most people seem to hate the name `counting_scope` because the “counting” is an implementation detail, there are arguments about whether it's really “scoping” anything, and the name doesn't really tell you what the type is *for*. The leading suggestion for a better name is to pick one that conveys that the type “groups together” or “keeps track of” “tasks”, “senders”, or “operations”. Examples of this scheme include `task_pool`, `sender_group`, and `task_arena`. We like the suggested pattern but seek LEWG's feedback on:
 - Should we choose `task` or `sender` to describe the thing being “grouped”? `task` feels friendlier, but might risk conveying that not all sender types are supported.
 - What word should we use to describe the “grouping”?
 - `pool` often means a pre-allocated group of resources that can be borrowed from and returned to, which isn't appropriate.
 - `group` is either the most generic word for a group of things, or an unrelated mathematical object.

- arena is used outside computing to mean a place where competitions happen, and within computing to refer to a memory allocation strategy.
- Something else?
- The name-part token was selected by analogy to `stop_token`, but it feels like a loose analogy. Perhaps `handle` or `ref` (short for reference) would be better. `ref` is nice for being short and accurate.
- The likely use of the `async_scope_token` concept will be to constrain algorithms that accept a sender and a token with code like the following:

```
template <sender Sender, async_scope_token Token>
void foo(Sender, Token);
```

We propose the token concept should be named `async_<new name of counting_scope> <new word for token>`. Assuming we choose `task_pool` and `ref`, that would produce `async_task_pool_ref`, which would look like this:

```
template <sender Sender, async_task_pool_ref Ref>
void foo(Sender, Ref);
```

- The `simple` prefix does not convey much about how `simple_counting_scope` is “simple”. Suggestions for alternatives include:
 - `fast` by analogy to the `fast`-prefixed standard integer types, which are so-named because they’re expected to be efficient.
 - `non_cancellable` to speak to what’s “missing” relative to `counting_scope`, however, `simple_counting_scope` does not change the cancellability of senders nested within it and we worry that this suggestion might convey that senders nested within a `non_cancellable` scope might somehow *lose* cancellability.

§ 7.1 `async_scope_token`

This is a concept that is satisfied by types that support nesting senders within themselves. It is primarily useful for constraining the arguments to `spawn()` and `spawn_future()` to give useful error messages for invalid invocations.

Since concepts don’t support existential quantifiers and thus can’t express “type `T` is an `async_scope_token` if there exists a sender, `s`, for which `t.nest(s)` is valid”, the `async_scope_token` concept must be parameterized on both the type of the token and the type of some particular sender and thus describes whether *this* token type is an `async_scope_token` in combination with *this* sender type. Given this limitation, perhaps the name should convey something about the fact that it is checking the relationship between two types rather than checking something about the scope’s type alone. Nothing satisfying comes to mind.

alternatives: `task_pool_ref`, `task_pool_token`, `task_group_ref`, `sender_group_ref`, `task_group_token`, `sender_group_token`, don’t name it and leave it as *exposition-only*

§ 7.2 `nest()`

This provides a way to build a sender that is associated with a “scope”, which is a type that implements and enforces some bookkeeping policy regarding the senders nested within it. `nest()` does not allocate state, call `connect`, or call `start`.

It would be good for the name to indicate that it is a simple operation (`insert`, `add`, `embed`, `extend` might communicate allocation, which `nest()` does not do).

alternatives: `wrap()`, `attach()`, `track()`, `add()`, `associate()`

§ 7.3 `spawn()`

This provides a way to start a sender that produces `void` and to associate the resulting async work with an async scope that can implement a bookkeeping policy that may help ensure the async work is complete before destroying any resources it is using. This allocates, connects, and starts the given sender.

It would be good for the name to indicate that it is an expensive operation.

alternatives: `connect_and_start()`, `spawn_detached()`, `fire_and_remember()`

§ 7.4 `spawn_future()`

This provides a way to start work and later ask for the result. This will allocate, connect, and start the given sender, while resolving the race (using synchronization primitives) between the completion of the given sender and the start of the returned sender. Since the type of the receiver supplied to the result sender is not known when the given sender starts, the receiver will be type-erased when it is connected.

It would be good for the name to be ugly, to indicate that it is a more expensive operation than `spawn()`.

alternatives: `spawn_with_result()`

§ 7.5 simple_counting_scope

A `simple_counting_scope` represents the root of a set of nested lifetimes.

One mental model for this is a semaphore. It tracks a count of lifetimes and fires an event when the count reaches 0.

Another mental model for this is block syntax. `{}` represents the root of a set of lifetimes of locals and temporaries and nested blocks.

Another mental model for this is a container. This is the least accurate model. This container is a value that does not contain values. This container contains a set of active senders (an active sender is not a value, it is an operation).

alternatives: `simple_async_scope`, `simple_task_pool`, `fast_task_pool`, `non_cancellable_task_pool`, `simple_task_group`, `simple_sender_group`

§ 7.6 counting_scope

Has all of the same behavior as `simple_counting_scope`, with the added functionality of cancellation; work nested in this scope can be asked to cancel *en masse* from the scope.

alternatives: `async_scope`, `task_pool`, `task_group`, `sender_group`

§ 7.6.1 counting_scope::join()

This method returns a sender that, when started, prevents new senders from being nested within the scope and then waits for the scope's count of outstanding senders to drop to zero before completing. It is somewhat analogous to `std::thread::join()` but does not block.

`join()` must be invoked, and the returned sender must be connected, started, and completed, before the scope may be destroyed so it may be useful to convey some of this importance in the name, although `std::thread` has similar requirements for its `join()`.

`join()` is the biggest wart in this design; the need to manually manage the end of a scope's lifetime stands out as less-than-ideal in C++, and there is some real risk that users will write deadlocks with `join()` so perhaps `join()` should have a name that conveys danger.

alternatives: `complete()`, `close()`

§ 8 Specification

§ 8.1 Async scope concepts

Add the following as a new subsection immediately after `[exec.utils.tfxcmplsig]`:

`std::execution::async_scope_association` `[exec.asyncscopeassociation.concept]`

- 1 The `async_scope_association<Assoc>` concept defines the requirements on an object of type `Assoc` that represents a possible association with an async scope object.

```
namespace std::execution {
template <class Assoc>
concept async_scope_association =
    semiregular<Assoc> &&
    requires(const Assoc& assoc) {
        { static_cast<bool>(assoc) } noexcept;
    };
}
```

- 2 `async_scope_association<Assoc>` is modeled only if `Assoc`'s copy and move operations are not potentially throwing.

`std::execution::async_scope_token` `[exec.asyncscopetoken.concept]`

- 1 The `async_scope_token<Token>` concept defines the requirements on an object of type `Token` that can be used to create associations between senders and an async scope.

```
namespace std::execution {
template <class Token>
concept async_scope_token =
    copyable<Token> &&
    requires(Token token) {
        { token.try_associate() } -> async_scope_association;
    };
}
```

```
    } &&  
}
```

- ² `async_scope_token<Token>` is modeled only if `Token`'s copy and move operations are not potentially throwing.
- ³ For a subexpression `snd`, let `Sndr` be `decltype((snd))` and let `sender<Sndr>` be true; `async_scope_token<Token>` is modeled only if, for an object, `token`, of type `Token`, the expression `token.wrap(snd)` is a valid expression and returns an object that satisfies `sender`.

§ 8.2 `execution::nest`

Add the following as a new subsection immediately after `[exec.stopped.as.error]`:

`std::execution::nest` `[exec.nest]`

- ¹ `nest` tries to associate a sender with an async scope such that the scope can track the lifetime of any async operations created with the sender.
- ² The name `nest` denotes a customization point object. For subexpressions `sndr` and `token`, let `Sndr` be `decltype((sndr))` and let `Token` be `decltype((token))`. If `sender<Sndr>` or `async_scope_token<Sender>` is false, the expression `nest(sndr, token)` is ill-formed.
- ³ Otherwise, the expression `nest(sndr, token)` is expression-equivalent to:
- TODO figure out how to express this in terms of `token.wrap(sndr)` and `token.try_associate()`
- ⁴ The evaluation of `nest(sndr, token)` may cause side effects observable via `token`'s associated async scope object.
- ⁵ Let the subexpression `out_sndr` denote the result of the invocation `nest(sndr, token)` or an object copied or moved from such, and let the subexpression `rcvr` denote a receiver such that the expression `connect(out_sndr, rcvr)` is well-formed. The expression `connect(out_sndr, rcvr)` has undefined behavior unless it creates an asynchronous operation (**`[async.ops]`**) that, when started:
- (5.1) — TODO: specify that starting `out_sndr` starts `sndr` unless `out_sndr` is an unassociated sender.

§ 8.3 `execution::spawn`

spec here

§ 8.4 `execution::spawn_future`

spec here

§ 8.5 `execution::simple_counting_scope`

spec here

§ 8.6 `execution::counting_scope`

spec here

§ 9 Acknowledgements

Thanks to Lewis Baker, Robert Leahy, Dmitry Prokoptsev, Anthony Williams, and everyone else who contributed to discussions leading to this paper.

Thanks to Andrew Royes for unwavering support for the development and deployment of Unifex at Meta and for recognizing the importance of contributing this paper to the C++ Standard.

Thanks to Eric Niebler for the encouragement and support it took to get this paper published.

§ 10 References

[Dahl72] O.-J. Dahl, E. W. Dijkstra, and C. A. R. Hoare. *Structured Programming*. Academic Press Ltd., 1972.

[`folly::coro`] `folly::coro`.

<https://github.com/facebook/folly/tree/main/folly/experimental/coro>

[`folly::coro::AsyncScope`] `folly::coro::AsyncScope`.

<https://github.com/facebook/folly/blob/main/folly/experimental/coro/AsyncScope.h>

[io_uring HTTP server] io_uring HTTP server.

https://github.com/facebookexperimental/libunifex/blob/main/examples/linux/http_server_io_uring_test.cpp

[libunifex] libunifex.

<https://github.com/facebookexperimental/libunifex/>

[P2079R2] Lee Howes, Ruslan Arutyunyan, Michael Voss. 2022-01-15. System execution context.

<https://wg21.link/p2079r2>

[P2300R7] Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. 2023-04-21. `std::execution`.

<https://wg21.link/p2300r7>

[P3296R2] Anthony Williams. let_async_scope.

<https://wg21.link/p3296r2>

[rsys] A smaller, faster video calling library for our apps.

<https://engineering.fb.com/2020/12/21/video-engineering/rsys/>

[unifex::v1::async_scope] unifex::v1::async_scope.

https://github.com/facebookexperimental/libunifex/blob/main/include/unifex/v1/async_scope.hpp

[unifex::v2::async_scope] unifex::v2::async_scope.

https://github.com/facebookexperimental/libunifex/blob/main/include/unifex/v2/async_scope.hpp